EDEESTE · SEÑALIZACIÓN DIGITAL

EDEEste.AdPlay

Documento de traspaso técnico
Qué es, cómo está construido, qué falta y por dónde seguir

Fecha	14 de octubre de 2026
Estado del código	Rama `Basics`, último commit `ad907ba` («Agregar invitaciones para que cada persona active su propia cuenta»)
Tecnología	.NET 10 (API en capas) · SQL Server · React 19 + TypeScript + Vite
Para quién es	La persona que continuará el desarrollo. Asume conocimiento de .NET y React, no del proyecto.
Cómo se elaboró	Revisión directa del código fuente, las migraciones, las pruebas y el README del repositorio, y análisis de dependencias con `dotnet list package --vulnerable` y `npm audit`.
Documento hermano	«Manual de usuario» (PDF aparte): cómo usar todo lo que existe hoy.

Contenido

1. Resumen ejecutivo
2. Propósito y funcionalidad esperada
3. Qué hace la aplicación hoy
4. Arquitectura
5. Backend por capas y referencia de la API
6. Base de datos
7. Seguridad: lo que ya existe
8. Panel de administración (frontend)
9. Reproductor y paquete local
10. Configuración, ejecución y despliegue
11. Pruebas y calidad
12. Revisión de seguridad: hallazgos
13. Deuda técnica y prácticas a mejorar
14. Lista de pendientes
15. Siguientes pasos recomendados
16. Anexos: decisiones, preguntas abiertas, glosario

> **Cómo leerlo.** Si es su primer día: lea las secciones 1 a 4 y 10 (una hora) y levante el proyecto. Si va a tocar el backend, lea la 5, 6 y 7. Si va a tocar el panel, la 8 y la 9. Las secciones 12 a 15 son el «qué sigue»: hallazgos de seguridad, deuda técnica, pendientes y un plan por fases.

1. Resumen ejecutivo

EDEEste.AdPlay es un sistema para administrar y reproducir anuncios digitales en las pantallas (televisores) de las sucursales de la empresa. Se administra desde una aplicación web («AdPlay Studio»); cada pantalla reproduce abriendo un enlace en su navegador, o con un paquete ZIP que se entrega al sistema de reproducción local que la empresa ya utiliza.

64 ENDPOINTS	**15** TABLAS PROPIAS	**4** MIGRACIONES	**98** PRUEBAS PASANDO
7 700 LÍNEAS C# (SIN PRUEBAS)	**10 300** LÍNEAS TS/TSX	**13** PÁGINAS DEL PANEL	**2** ROLES

Dónde estamos

- **Implementado:** las fases 1 (imprescindibles), 2 (supervisión) y 3 (horarios, usuarios y roles, auditoría, historial y reportes) del plan de trabajo; el panel completo con el diseño nuevo (tema claro/oscuro); el reproductor web con caché sin conexión; el registro de la primera cuenta; las invitaciones para activar cuentas; y la foto de perfil.
- **Verificado:** compila, `dotnet test` pasa 98 de 98, y `npm run lint` y `npm run build` pasan sin errores. `npm audit` no reporta vulnerabilidades.
- **No verificado todavía:** no hay pruebas de integración de la API ni pruebas del frontend. La migración `Invitaciones` y el flujo de invitación completo se probaron con pruebas unitarias y con el panel contra datos simulados; **falta una prueba de punta a punta contra un SQL Server real**.
- **No existe:** automatización de la entrega del paquete local (hoy el ZIP se descarga y se copia a mano a la carpeta del sistema de reproducción que la empresa ya usa, y no está confirmado que ese sistema entienda el `manifest.json`), envío de correos, recuperación de contraseña, CI/CD, contenedores ni despliegue productivo.

Lo más importante que sigue

1. Corregir la dependencia transitiva con vulnerabilidad **ALTA** (`System.Security.Cryptography.Pkcs` 9.0.11) — sección 12, hallazgo S-01.
2. Probar de punta a punta la migración y el flujo de invitaciones contra SQL Server real.
3. Endurecer la sesión (revocación real, tope absoluto) y agregar cabeceras de seguridad HTTP.
4. Montar CI (compilar, probar, lint, auditar dependencias) y definir el despliegue.
5. Cerrar con negocio/TI las preguntas abiertas de la «Fase 0» (sección 16.2): confirmar que el sistema de reproducción local existente acepta el paquete y decidir si se automatiza su entrega.

2. Propósito y funcionalidad esperada

2.1 Qué problema resuelve

Hoy, publicar un anuncio en un televisor de sucursal es un proceso manual. AdPlay centraliza el contenido: se sube una vez, se organiza en listas, se *publica* una versión y las pantallas que correspondan la reproducen —sin que alguien tenga que ir a cada equipo—. También permite saber, en un solo lugar, qué pantallas están encendidas, si tienen la versión correcta y qué están mostrando.

El usuario de negocio previsto es **Comunicaciones Internas**: administra anuncios, listas y pantallas y publica cambios de forma centralizada, sin conectar memorias USB a los televisores. La empresa opera en República Dominicana: la zona horaria predeterminada es `America/Santo_Domingo` y las sucursales sembradas son **Megacentro** y **Sabana Larga**.

> **Documento de referencia fuera del repositorio.** El plan por fases, los criterios de aceptación y las preguntas de la «Fase 0» salen de la guía de diseño original `EDEEste.AdPlay_Guia_Arquitectura.pdf` (14 páginas, 12 de octubre de 2026), que el README cita pero **no está versionada** junto al código. Archívela en el repositorio (por ejemplo, en `docs/`). Este documento resume lo que el código implementa y lo compara con esa guía en la sección 2.5.

2.2 Actores

Actor	Qué hace	Cómo entra
Administrador	Todo: estructura (sucursales, grupos, pantallas), asignaciones, **publicar y retirar versiones**, usuarios, invitaciones y auditoría. Además puede hacer lo del Operador.	Usuario y contraseña en el panel
Operador	Prepara contenido: sube multimedia, crea anuncios, arma listas y exporta paquetes. No publica, no asigna, no administra. Puede consultar supervisión e historial.	Usuario y contraseña en el panel
Pantalla (dispositivo)	Pide su manifiesto, descarga los medios que le tocan, los reproduce y envía latidos. Solo ve lo de su propia asignación.	Token de dispositivo en la cabecera `X-Token-Dispositivo`
Persona invitada	Activa su cuenta con un enlace de un solo uso y elige su contraseña.	Enlace `/activar?token=…`

2.3 Flujo de negocio de punta a punta

1. **Estructura.** El administrador registra sucursales y grupos (anidables dentro de una sucursal) y registra cada pantalla en un grupo, eligiendo su modo: navegador web o paquete local.
2. **Enlace de la pantalla.** Genera el enlace/token del dispositivo (se muestra una sola vez) y lo abre en el navegador del televisor.
3. **Contenido.** Un operador sube imágenes y videos a la biblioteca y los convierte en *anuncios* (título, archivo, duración predeterminada).

4. **Listas.** Arma *listas de reproducción*: anuncios ordenados, con duración personalizada opcional.
5. **Publicación.** El administrador *publica* una versión: una instantánea inmutable (v1, v2…). Editar la lista después no cambia lo ya publicado.
6. **Asignación.** El administrador asigna la lista a una sucursal, grupo o pantalla, con vigencia (fechas) y franjas semanales opcionales (días, horas, zona horaria).
7. **Reproducción.** La pantalla consulta su manifiesto cada 30 s; si hay versión nueva, descarga los medios, verifica su hash, cambia de versión completa y reproduce en bucle. Cada consulta envía un latido (versión cargada, error, anuncio y segundo actual).
8. **Supervisión.** El panel compara la versión que cada pantalla *debería* reproducir con la que informó, y muestra en línea/desconectada, desactualizada, con error y qué anuncio está en pantalla.

2.4 Matriz de permisos

Política	Quién	Operaciones
`Lectura`	Cualquier usuario autenticado	Consultar todo: supervisión, estructura, multimedia, listas, historial; ver el contenido de los medios; cambiar su propia contraseña y foto.
`Contenido`	Administrador y Operador	Subir y archivar multimedia; crear y editar anuncios y listas; generar y descargar paquetes ZIP.
`Administracion`	Solo Administrador	Sucursales, grupos, pantallas y sus tokens; asignaciones; **publicar y retirar** versiones; usuarios e invitaciones; auditoría.

2.5 Cumplimiento de los criterios de aceptación de la guía

La guía original fija siete criterios mínimos. Estado según el código (sin una prueba de aceptación con televisores reales; ver sección 11):

Criterio de la guía	Estado	Evidencia / observación
Una persona autorizada puede subir una imagen, añadirla a una lista y publicarla sin acceder a SQL Server.	CUMPLE	Flujo completo en el panel. Publicar exige rol Administrador (el Operador prepara, no publica).
Una pantalla web recibe la versión nueva en el intervalo esperado y reproduce en bucle.	CUMPLE	Consulta cada 30 s con ETag; cambio de versión completa. Falta probarlo en el navegador de los televisores reales.
Desactivar una pantalla web muestra un mensaje/animación en lugar de una página vacía.	CUMPLE	«No hay contenido disponible» con detalle «Pantalla deshabilitada».
El paquete local incluye los archivos y el manifiesto correctos; el hash detecta archivos dañados.	PARCIAL	El backend lo genera y verifica hashes. **No está confirmado** que el sistema local existente interprete ese `manifest.json` ni cómo detecta archivos nuevos.
Una desconexión no deja al reproductor web sin contenido si ya tenía una versión válida.	CUMPLE	Manifiesto y medios en caché; arranque sin conexión.
Los operadores sin permiso de administración no pueden publicar ni administrar usuarios.	CUMPLE	Políticas en la API. **Sin prueba automatizada** que lo garantice (hallazgo en sección 11).

Criterio de la guía	Estado	Evidencia / observación
La base rechaza asignaciones ambiguas y posiciones/versiones duplicadas.	PARCIAL	Posiciones y versiones: índices únicos en la base. Destino único: `CHECK`. Las asignaciones *ambiguas* (fechas y horario coincidentes) las rechaza el servicio de aplicación, no la base.

Otras exigencias de la guía que quedan abiertas

- **Análisis antimalware de los archivos subidos** (sección 11 de la guía) — no existe (hallazgo S-07).
- **Recuperación segura de contraseña** — no existe; solo restablece un administrador.
- **Plan de respaldo y restauración probada** de SQL Server y de la carpeta de medios, y política de retención — no definido.
- **Historial técnico de estados** (`HistorialEstadoPantalla`, opcional en la guía) — no implementado.
- **Compatibilidad del navegador de los televisores** (pantalla completa, suspensión, códecs) — Fase 0, sin verificar.

Toda la API exige usuario autenticado por defecto (`FallbackPolicy`); solo se abre lo que se marca `AllowAnonymous`: inicio de sesión, registro inicial, renovación, consulta/activación de invitación y los tres endpoints del reproductor.

3. Qué hace la aplicación hoy

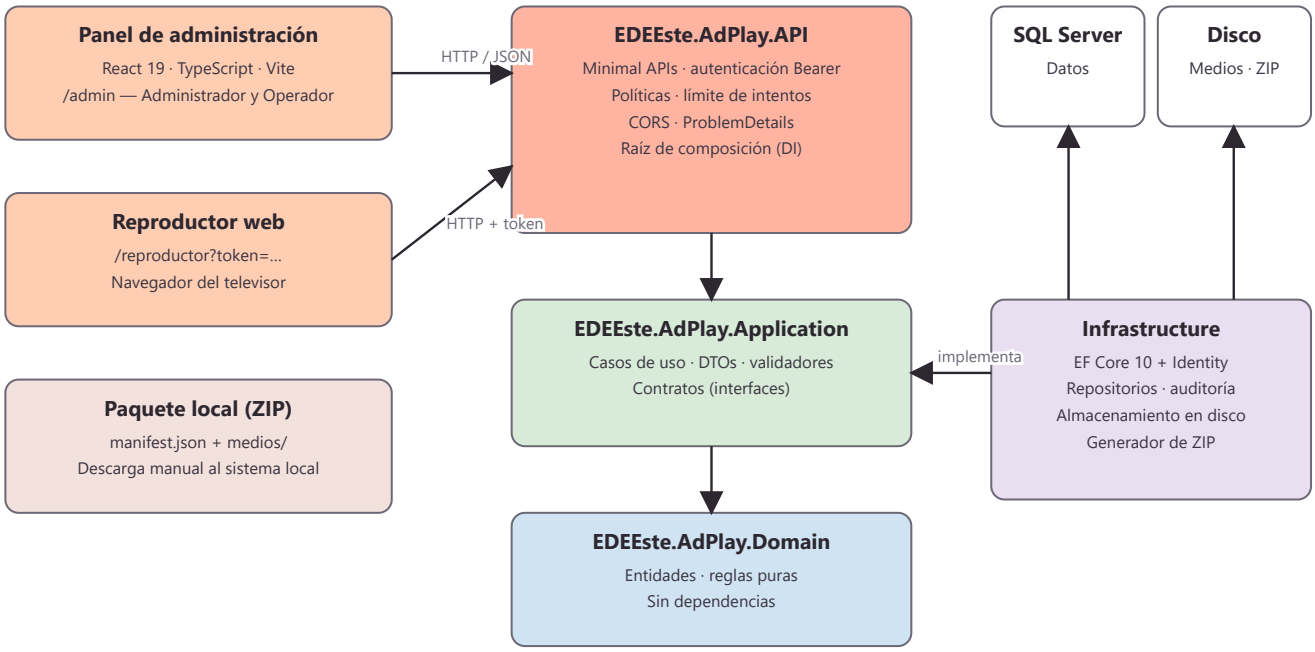
Inventario por módulo. COMPLETO = funciona de punta a punta en el código; PARCIAL = funciona con limitaciones conocidas; NO EXISTE = previsto pero sin implementar.

Módulo	Qué hace	Estado
Acceso	Inicio de sesión (con límite de intentos y bloqueo temporal); registro de la primera cuenta (solo mientras no haya usuarios); activación de cuenta por invitación; renovación automática del token; cierre de sesión en el cliente.	COMPLETO
Supervisión	Tablero con estado de cada pantalla (en línea, desconectada, sin datos, desactualizada, con error, deshabilitada), filtros, búsqueda, actualización automática cada 30 s y exportación a CSV.	COMPLETO
Sucursales y grupos	Alta y edición de sucursales (código, nombre, zona horaria) y de grupos anidables dentro de una sucursal; el sistema impide ciclos y mover grupos con subgrupos entre sucursales. No se eliminan: se desactivan.	COMPLETO
Pantallas	Registro, edición, habilitar/deshabilitar, modo (web/paquete local), generar y rotar el enlace del dispositivo, revocarlo, vista en vivo con el anuncio actual y el avance estimado.	COMPLETO
Multimedia y anuncios	Subida con validación (extensión, MIME, firma, tamaño) y progreso; biblioteca con vista previa; archivado; deduplicación por hash; creación y edición de anuncios.	PARCIAL sin miniatura de video; duración de video manual; sin ancho/alto
Listas y publicación	Listas con orden por arrastre y duración personalizada; vista previa en bucle; guardar; publicar versión (inmutable); retirar versión; asignaciones con vigencia y horarios; sección de versiones con paquete ZIP.	COMPLETO sin control de concurrencia
Historial	Publicaciones y exportaciones de todas las listas, con filtros, tarjetas de resumen y descarga de paquetes.	COMPLETO
Usuarios	Listado, cambio de rol, activar/desactivar, restablecer contraseña, crear con contraseña inicial o **invitar** (enlace de un solo uso, vence a los 7 días), reenviar/anular invitaciones. Protege que siempre quede un administrador activo.	PARCIAL el enlace se comparte a mano; no hay correo
Auditoría	Registro inmutable de acciones con filtros por fecha, usuario, acción y elemento; marca las acciones de riesgo; resumen en tarjetas.	COMPLETO
Mi cuenta	Cambio de contraseña, foto de perfil con recorte, tema claro/oscuro, cierre de sesión.	COMPLETO
Reproductor web	Manifiesto con ETag, descarga y verificación de hash de cada medio, caché sin conexión, cambio de versión atómico, reporte de errores y latidos con anuncio y segundo.	COMPLETO

Módulo	Qué hace	Estado
Paquete local (ZIP)	El backend genera el ZIP (`manifest.json` , `version.txt` , `medios/`) con verificación de hash y lo ofrece para descarga; la persona lo copia a la carpeta del sistema de reproducción local que la empresa ya usa. **No hay automatización ni confirmación de compatibilidad** con ese sistema.	**PARCIAL** entrega manual; compatibilidad por confirmar
Notificaciones por correo	Invitaciones, recuperación de contraseña, alertas de desconexión.	**NO EXISTE**
Sincronización en tiempo real	SignalR / agente local. El plan lo condiciona a respuestas de la Fase 0 sin confirmar. Hoy: consulta periódica.	**NO EXISTE**

4. Arquitectura

Clean Architecture / Onion: las dependencias apuntan hacia el centro. El panel y el reproductor web son el *mismo* proyecto React (`EDEEste.AdPlay.Presentation`), con rutas separadas (`/admin/*`, `/reproductor`, `/iniciar-sesion`, `/registro`, `/activar`).



Las dependencias apuntan hacia el centro (Domain). Infrastructure implementa los contratos que define Application.

Figura 1. Componentes y sentido de las dependencias.

4.1 Proyectos de la solución

Proyecto	Responsabilidad	Depende de
`EDEEste.AdPlay.Domain`	Entidades, enums y reglas de negocio puras (resolución de asignaciones, horarios, jerarquía de grupos). Sin dependencias externas. 33 archivos.	—
`EDEEste.AdPlay.Application`	Casos de uso (servicios), DTOs, validadores, contratos (interfaces) y excepciones de aplicación. 70 archivos.	Domain
`EDEEste.AdPlay.Infrastructure`	EF Core 10 / SQL Server, ASP.NET Core Identity, repositorios, almacenamiento en disco, generador de ZIP, auditoría. 41 archivos.	Application, Domain
`EDEEste.AdPlay.API`	Endpoints (Minimal APIs), autenticación, políticas, CORS, límite de intentos, manejo de errores. Es la raíz de composición. 17 archivos.	Application, Infrastructure
`EDEEste.AdPlay.Presentation`	Panel y reproductor web. React 19, React Router 7, TypeScript 6, Vite 8. Sin librerías de UI ni de estado: solo React y CSS propio.	— (HTTP)

Proyecto	Responsabilidad	Depende de
`tests/EDEEste.AdPlay.Tests`	xUnit. Pruebas de Domain, de servicios de Application con dobles manuales y del modelo de EF Core (sin base de datos).	Domain, Application, Infrastructure

4.2 Convenciones que debe respetar

- **Idioma:** el lenguaje de negocio (entidades, propiedades, tablas, casos de uso, mensajes) está en español; los términos técnicos comunes, en inglés (`Entities`, `Endpoints`, `Services`).
- **Un tipo por archivo** en C#. Los DTO se agrupan por área en `DTOs/*Dtos.cs`.
- **Los endpoints nunca exponen entidades** ni consultan `ContextoPublicidad`: llaman a los servicios de `Application/CasosDeUso`.
- **Reglas que no necesitan base de datos viven en Domain** y tienen pruebas. Los casos de uso orquestan: cargan, llaman al dominio, persisten.
- **Constantes compartidas** (longitudes de texto, límites de duración, zona horaria) están en `Domain/Common`; las usan validaciones y esquema. Cambiar una longitud requiere migración.
- **Errores:** los servicios lanzan `ExcepcionValidacion` (400), `ExcepcionNoEncontrado` (404), `ExcepcionConflicto` (409) o `ExcepcionNoAutorizado` (401); `ManejadorExcepciones` los traduce a `ProblemDetails` y nunca devuelve trazas.
- **Fechas siempre en UTC** (`…Utc`); se convierten a hora local solo en la interfaz.
- **Medios inmutables y fuera de SQL Server.** La clave de almacenamiento la genera el sistema; lo referenciado no se borra, se archiva.
- **Publicaciones inmutables.** Editar una lista no altera una versión publicada.
- **Auditoría:** cada caso de uso que cambia algo llama a `IRegistroAuditoria.Registrar(...)` *antes* de guardar; se persiste en la misma transacción. Nunca incluir contraseñas, tokens ni secretos en el detalle.
- **Usuarios:** el Dominio solo conoce ids de usuario (`int?`). Identity vive en Infrastructure y se usa mediante `IGestorUsuarios` e `IUsuarioActual`.

4.3 Cómo agregar una funcionalidad (receta)

1. Entidad o regla en `Domain`, con su prueba.
2. Contrato en `Application/Contratos`, DTOs, validador y caso de uso en `Application/CasosDeUso`; regístrelo en `Application/InyeccionDependencias.cs`.
3. Configuración de EF y repositorio en `Infrastructure`; regístrelo en `Infrastructure/InyeccionDependencias.cs`.
4. Endpoint en `API/Endpoints` y mapeo en `Program.cs`, con su política de autorización.
5. Migración y, si aplica, tipos y servicio en `Presentation/src/{types,services}` y la página.

5. Backend por capas y referencia de la API

5.1 Domain

Elemento	Reglas que encapsula
`Sucursal`, `GrupoPantalla`, `Pantalla`	Jerarquía sucursal → grupo (anidable) → pantalla. La sucursal de una pantalla se deduce de su grupo (no se duplica). `CalcularEstadoOperativo`: EnLinea / Desconectada / SinDatos según la ventana sin latido.
`JerarquiaGrupos`	Cadena de ancestros de un grupo, detección de ciclos.
`ArchivoMultimedia`, `Anuncio`	Medio inmutable (hash SHA-256, clave generada); anuncio = medio + título + duración predeterminada.
`ListaReproduccion`, `ElementoListaReproduccion`	Lista editable; `ReemplazarElementos` asigna posiciones consecutivas desde 1 y valida duraciones (1–3600 s).
`PublicacionLista`, `ElementoPublicacion`	Instantánea inmutable con número de versión; identificador de versión `{Id}.{NumeroVersion}` que reportan y comparan los reproductores.
`AsignacionLista`, `ReglaHorario`	Exactamente un destino; vigencia por fechas (extremo nulo = abierto); franjas semanales con zona horaria; detección de solapamiento de fechas y de horarios entre asignaciones del mismo destino.
`ResolutorAsignaciones`	Prioridad: **pantalla** → grupo más cercano hacia arriba → **sucursal**; desempata por la más reciente. Solo considera asignaciones activas ahora (vigencia y horario) con lista activa.
`Exportacion`, `EstadoPantalla`, `RegistroAuditoria`, `Invitacion`	Paquete generado; último latido y reproducción en curso (1:1 con pantalla); quién hizo qué; invitación con hash de token y estado calculado (Pendiente, Vencida, Activada, Revocada).

5.2 Application (casos de uso)

Servicio	Responsabilidad
`ServicioAutenticacion`	Validar credenciales y auditar (incluido el fallo); registro inicial con exclusión mutua; releer sesión al renovar.
`ServicioUsuarios`	Crear, actualizar rol/estado, restablecer y cambiar contraseña; impide desactivarse a sí mismo y dejar el sistema sin administrador activo.
`ServicioInvitaciones`	Crear (anula las abiertas del mismo correo), listar, anular, consultar y activar. Token de 32 bytes aleatorios; solo se guarda su SHA-256; vigencia 7 días.
`ServicioFotoPerfil`	Validar imagen (firma real), guardar y servir la foto de cada usuario.
`ServicioSucursales` / `ServicioGruposPantallas`	Alta y edición con reglas de jerarquía y unicidad de código.
`ServicioPantallas`	Registro, edición, generar/rotar/revocar token (se guarda solo el hash; el token en claro se devuelve una vez).

Servicio	Responsabilidad
`ServicioMultimedia` / `ServicioAnuncios`	Carga con validación y deduplicación por hash (si ya existe, se reutiliza y se restaura si estaba archivado); archivar; anuncios.
`ServicioListasReproduccion`	Crear y reemplazar datos y elementos de la lista.
`ServicioAsignaciones`	Crear/editar/eliminar con rechazo de asignaciones ambiguas (mismo destino, fechas y horario coincidentes).
`ServicioPublicaciones`	Validar que la lista sea publicable (activa, con anuncios, anuncios activos, medios no archivados y presentes en almacenamiento) y crear la versión; retirar.
`ServicioExportaciones`	Generar el ZIP de una publicación vigente (síncrono, dentro de la petición) y servir la descarga.
`ServicioReproductor`	Autenticar por token, resolver la publicación aplicable, servir manifiesto y medios (solo los de la versión que le toca), registrar latidos.
`ServicioReportes` / `ServicioHistorial` / `ServicioAuditoria`	Reporte de estado esperado vs. reportado; historial y resúmenes paginados; consulta de auditoría.

5.3 Infrastructure

- **Persistencia:** `ContextoPublicidad` (hereda de `IdentityDbContext<UsuarioAplicacion, RolAplicacion, int>`). Configuraciones por entidad en `Persistencia/Configuraciones`; convertidor de fechas a UTC con precisión 3; enums como texto salvo `DiasSemana` (flags, numérico); tabla de historial de migraciones `HistorialMigraciones`.
- **Identidad:** `GestorUsuarios` implementa `IGestorUsuarios` sobre `UserManager`; `InicializadorSeguridad` crea roles y, opcionalmente, el administrador inicial; mensajes de error de Identity en español (`DescriptorErroresIdentidadEs`).
- **Almacenamiento:** `AlmacenamientoMultimediaDisco` guarda bajo una raíz controlada, impide salir de ella (`..`), escribe a archivo temporal y mueve, calcula el SHA-256 al guardar y nunca sobrescribe.
- **Paquetes:** `GeneradorPaquetesZip` verifica el hash de cada medio al empaquetar y el del paquete tras guardarlo.
- **Auditoría:** `RegistroAuditoriaEf` acumula en el contexto y se guarda con la misma unidad de trabajo que el cambio.

5.4 API: referencia de endpoints

Prefijo `/api`. Política: **L** = Lectura, **C** = Contenido, **A** = Administracion, **Anón.** = anónimo, **RL** = con límite de intentos (10/min por IP). Respuestas de error en `ProblemDetails` (RFC 9457).

Método	Ruta	Política	Descripción
Autenticación y cuenta			
POST	`/auth/iniciar-sesion`	Anón. RL	Credenciales → `accessToken`, `refreshToken`, `expiresIn`.
GET	`/auth/registro-inicial`	Anón.	¿Está disponible el registro de la primera cuenta?

Método	Ruta	Política	Descripción
POST	`/auth/registro-inicial`	Anón. RL	Crea al primer administrador (409 si ya hay usuarios).
POST	`/auth/renovar`	Anón. RL	Cambia un token de renovación por nuevos tokens.
GET	`/auth/sesion`	L	Datos de la sesión actual.
POST	`/auth/invitacion`	Anón. RL	Consulta una invitación (token en el cuerpo).
POST	`/auth/activar`	Anón. RL	Activa la cuenta y devuelve sesión iniciada.
POST	`/auth/cambiar-clave`	L	Cambia la contraseña propia.
PUT · DELETE	`/auth/foto`	L	Sube (multipart, campo `archivo`) o quita la foto propia.
GET	`/usuarios/{id}/foto`	L	Foto de un usuario (caché por versión `?v=`).
Usuarios e invitaciones			
GET · POST	`/usuarios`	A	Listar; crear con contraseña inicial.
PUT	`/usuarios/{id}`	A	Nombre, rol y estado.
POST	`/usuarios/{id}/restablecer-clave`	A	Asigna una contraseña nueva y levanta el bloqueo.
GET · POST	`/invitaciones`	A	Listar abiertas; crear (devuelve el token una sola vez).
DELETE	`/invitaciones/{id}`	A	Anular.
Estructura			
GET · POST · PUT	`/sucursales`, `/sucursales/{id}`	L / A	Consultar (L); crear y editar (A).
GET · POST · PUT	`/grupos-pantallas`, `/{id}`	L / A	Filtro `sucursalId`.
GET	`/pantallas`, `/pantallas/{id}`	L	Filtros `sucursalId`, `grupoPantallaId`.
POST · PATCH	`/pantallas`, `/pantallas/{id}`	A	Registrar; actualizar (parcial: `null` = no cambiar).
POST · DELETE	`/pantallas/{id}/token`	A	Generar/rotar (devuelve token en claro una vez); revocar.
Contenido			
GET	`/multimedia`, `/{id}`, `/{id}/contenido`	L	Listar (`incluirArchivados`), detalle, contenido con `Range` y ETag.

Método	Ruta	Política	Descripción
POST · PATCH	`/multimedia`, `/multimedia/{id}`	C	Subir (multipart; límite por tamaño); archivar/restaurar.
GET · POST · PUT	`/anuncios`, `/{id}`	L / C	Consultar; crear y editar.
GET · POST · PUT	`/listas-reproduccion`, `/{id}`	L / C	Consultar; crear y editar (reemplaza elementos).
Publicación y asignación			
POST	`/listas-reproduccion/{id}/publicar`	A	Crea una versión inmutable.
GET	`/listas-reproduccion/{id}/publicaciones`	L	Versiones de una lista.
GET · POST	`/publicaciones/{id}`, `/{id}/retirar`	L / A	Detalle; retirar versión.
GET · POST	`/publicaciones/{id}/exportaciones`	L / C	Listar paquetes; generar paquete ZIP.
GET	`/exportaciones/{id}/descarga`	C	Descarga del ZIP (audita).
GET · POST · PUT · DELETE	`/asignaciones`, `/{id}`	L / A	Filtro `listaReproduccionId`; crear, editar, eliminar.
Consultas			
GET	`/reportes/estado-pantallas`	L	Esperado vs. reportado, con reproducción en curso.
GET	`/historial/resumen`, `/publicaciones`, `/exportaciones`	L	Paginados con filtros.
GET	`/auditoria`, `/auditoria/resumen`	A	Paginado con filtros.
Reproductor (cabecera `X-Token-Dispositivo`)			
GET	`/reproductor/manifiesto`	Anón.	Manifiesto vigente; ETag y 304.
GET	`/reproductor/medios/{id}`	Anón.	Medio (solo si está en la versión aplicable); `Range`.
POST	`/reproductor/latido`	Anón.	Versión cargada, error y reproducción en curso.

6. Base de datos

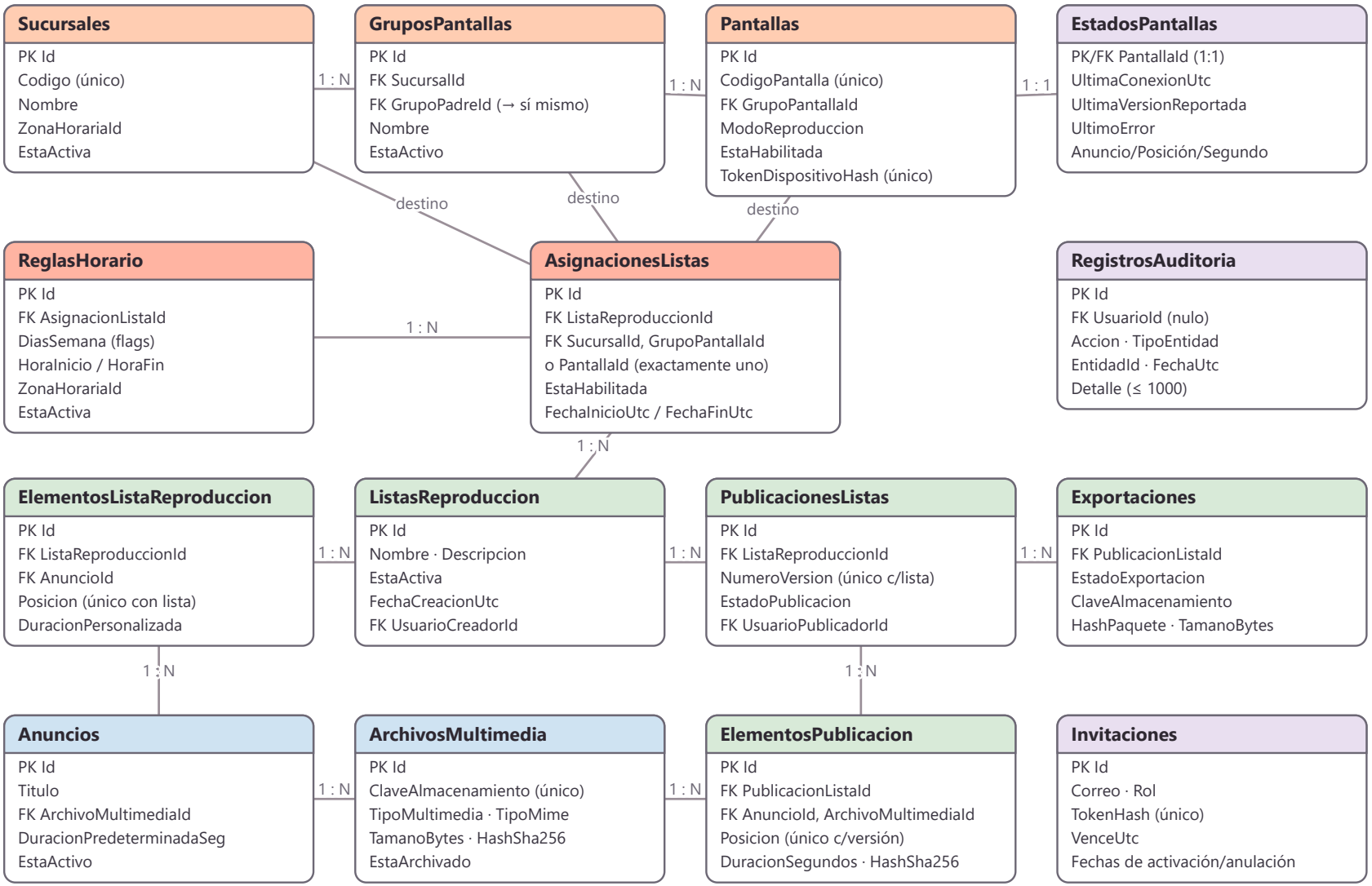
SQL Server, acceso con EF Core 10 (code first). El esquema vive en `Infrastructure/Persistencia`; las migraciones, en `Persistencia/Migraciones`. Hay **15 tablas propias**, las **7 tablas de Identity** (renombradas al español) y la tabla `HistorialMigraciones`.

6.1 Decisiones de modelado que conviene conocer

- **Los usuarios no se eliminan** (la auditoría y varias tablas los referencian con FK *Restrict*): se desactivan (`EstaActivo`). Tampoco hay borrado de sucursales, grupos, pantallas, listas ni anuncios desde la aplicación: se desactivan o archivan.
- **Enums como texto** (`ModoReproduccion`, `TipoMultimedia`, `EstadoPublicacion`, `EstadoExportacion`, `AccionAuditoria`): legibles en la base, pero **renombrar un valor del enum rompe los datos existentes**. Agregue valores nuevos, no renombre.
- `DiasSemana` **es un enum de banderas guardado como número** (domingo=1 … sábado=64) a propósito; el CHECK limita 1–127.
- **Fechas:** todo `datetime2(3)` en UTC mediante `ConvertidorFechaUtc`.
- **El estado de una invitación no se guarda**: se calcula con sus fechas (`EstadoEn`).
- **La clave de almacenamiento** de medios es `medios/AAAA/MM/{guid}.{ext}`; paquetes `exportaciones/{guid}.zip`; fotos `perfiles/{usuarioId}/{guid}.{ext}`.

El diagrama de la página siguiente muestra las tablas propias y sus relaciones; después se detalla cada una.

6.2 Diagrama del modelo de datos



Color: crema = estructura · naranja = asignación y horarios · verde = listas y versiones · azul = medios · lila = soporte (estado, auditoría, invitaciones).

Figura 2. Tablas propias y relaciones (se muestran las columnas más relevantes, no todas). Las 7 tablas de Identity no se dibujan.

6.3 Tablas propias

Tabla	Propósito	Restricciones e índices destacados
`Sucursales`	Sedes. Semilla: **MEGACENTRO** y **SABANA-LARGA**.	Único `Codigo`.
`GruposPantallas`	Ubicaciones dentro de una sucursal; autorreferencia `GrupoPadreId`.	FK Restrict. Que el padre sea de la misma sucursal y que no haya ciclos se valida en Application, no en la base.
`Pantallas`	Televisores/reproductores, con modo y `TokenDispositivoHash` (SHA-256 hex).	Único `CodigoPantalla`; único filtrado en `TokenDispositivoHash` (no nulo).
`EstadosPantallas`	Último latido, versión reportada, error y reproducción en curso (1:1).	PK = FK `PantallaId`, cascada.
`ArchivosMultimedia`	Metadatos del medio (el binario está en disco).	`CK TamanoBytes > 0`; único `ClaveAlmacenamiento`; índice en `HashSha256`.
`Anuncios`	Medio + título + duración predeterminada.	`CK DuracionPredeterminadaSegundos > 0`; FK Restrict al medio.
`ListasReproduccion`	Listas editables.	—
`ElementosListaReproduccion`	Anuncios de una lista, en orden.	Único (`ListaReproduccionId`, `Posicion`); cascada con la lista.
`AsignacionesListas`	Lista → sucursal | grupo | pantalla, con vigencia.	`CK` exactamente un destino; `CK` fin > inicio; índice (inicio, fin).
`ReglasHorario`	Franjas semanales de una asignación.	`CK` días entre 1 y 127 (flags); cascada con la asignación.
`PublicacionesListas`	Versiones inmutables de una lista.	Único (`ListaReproduccionId`, `NumeroVersion`); `CK NumeroVersion > 0`.
`ElementosPublicacion`	Instantánea de cada anuncio de la versión (medio, orden, duración, hash).	Único (publicación, posición); todo Restrict (histórico).
`Exportaciones`	Paquetes ZIP generados de una publicación.	FK Restrict a la publicación.
`RegistrosAuditoria`	Quién hizo qué y cuándo.	Índices por `FechaUtc` y (`TipoEntidad`, `EntidadId`); FK Restrict a usuarios.
`Invitaciones`	Invitaciones para activar cuenta; guarda solo el hash del token.	Único `TokenHash`; índice por `Correo`; FKs a usuarios (creador y activado).

6.4 Migraciones

Migración	Contenido
`20261012155334_Inicial`	13 tablas: sucursales, grupos, pantallas y su estado, medios, anuncios, listas y elementos, asignaciones y horarios, publicaciones y sus elementos, exportaciones; más la semilla de sucursales.
`20261012193352_FaseTres`	Identidad (usuarios, roles y sus 5 tablas de apoyo, renombradas al español), `RegistrosAuditoria` y las claves foráneas hacia los usuarios.
`20261014012934_PanelNuevo`	Columnas de foto de perfil en `Usuarios` y de reproducción en curso en `EstadosPantallas`.
`20261014093741_Invitaciones`	Tabla `Invitaciones` y sus FKs. **Pendiente de probar contra SQL Server real.**

```
dotnet ef migrations add NombreDelCambio --project EDEEste.AdPlay.Infrastructure --startup-project
EDEEste.AdPlay.API --output-dir Persistencia/Migraciones
dotnet ef database update --project EDEEste.AdPlay.Infrastructure --startup-project EDEEste.AdPlay.API
```

> En `Development` las migraciones se aplican solas al iniciar la API (`BaseDatos:AplicarMigracionesAlIniciar`). En producción no: genere un script (`dotnet ef migrations script --idempotent`) y apliquelo como paso controlado del despliegue. Hoy el repositorio no incluye ese procedimiento.

7. Seguridad: lo que ya existe

Esta sección describe los controles implementados. Los problemas y brechas están en la sección 12.

Tema	Qué hay
Autenticación	ASP.NET Core Identity valida las credenciales; la sesión viaja como token **Bearer** (`AddBearerToken`, tokens protegidos con Data Protection, sin estado en servidor). Acceso 30 min, renovación 12 h (configurable).
Autorización	Política por defecto *FallbackPolicy*: usuario autenticado. Tres políticas: Lectura, Contenido, Administracion. Dos roles: Administrador y Operador.
Contraseñas	Mínimo 10 caracteres con mayúscula, minúscula y número; hash de Identity (PBKDF2). Bloqueo de la cuenta por 10 min tras 5 intentos fallidos.
Enumeración de cuentas	Mismo mensaje para «no existe», «clave incorrecta» e «inactiva»; si el usuario no existe se hace un hash de todos modos para igualar tiempos.
Límite de intentos	10/min por IP en inicio de sesión, registro inicial, renovación, consulta y activación de invitación (`FixedWindowRateLimiter`).
Tokens de dispositivo	32 bytes aleatorios en Base64Url; se guarda solo el SHA-256; el token en claro se muestra una vez; rotar invalida el anterior; revocar deja la pantalla sin acceso.
Aislamiento del reproductor	Una pantalla solo puede descargar medios que pertenezcan a la versión que le corresponde ahora: no ve contenido de otras sucursales ni sin publicar.
Invitaciones	Token de un solo uso en el *cuerpo* de la petición (no en la URL del API), hash en base de datos, vencimiento a 7 días, anulación al invitar de nuevo, mensaje genérico ante token no válido, exclusión mutua al activar.
Archivos subidos	Lista blanca: `.jpg .jpeg .png .webp .mp4 .webm`; deben coincidir extensión, MIME declarado y **firma real** del contenido; límite por tipo (20 MB imagen, 500 MB video, 2 MB foto); clave de almacenamiento generada por el sistema (nunca el nombre original); sin salida de la raíz con `..`.
Integridad	SHA-256 de cada medio al guardar; el reproductor verifica el hash antes de usarlo; el generador de ZIP verifica cada medio y el paquete.
Errores	`ProblemDetails` sin trazas; los 500 se registran en servidor con `traceId`.
Auditoría	Se guarda en la misma transacción que el cambio; no se puede editar ni borrar desde la aplicación.
Secretos	Cadena de conexión y administrador inicial en User Secrets / variables de entorno; `.env.development` sin secretos; `App_Data/` ignorado por git.
CORS	Lista blanca de orígenes por configuración (`Cors:OrigenesPermitidos`); en desarrollo, `http://localhost:5173`.
Front	React escapa por defecto y no se usa `dangerouslySetInnerHTML` ni `eval`; el token de sesión vive en `sessionStorage` (se pierde al cerrar la pestaña).

8. Panel de administración (frontend)

8.1 Stack y estructura

React 19, React Router 7, TypeScript 6 y Vite 8. **No hay librería de componentes ni de estado**: el estado se maneja con hooks propios y contextos; los estilos son CSS propio con variables (tokens) y un archivo por página. Fuentes: Charis SIL (títulos) y Sansation (texto), cargadas desde Google Fonts.

Carpeta (`src/`)	Contenido
`pages/`	Páginas públicas (`IniciarSesionPage`, `RegistroInicialPage`, `ActivarCuentaPage`, `MarcoAcceso`) y `admin/` (supervisión, sucursales, pantallas, multimedia, listas, historial, usuarios, auditoría, cuenta) y `reproductor/`.
`components/`	`LayoutAdmin` (barra superior y menú), `RutaProtegida`, `Ventanas` (Dialogo, Cajon, Confirmacion), `Comunes` (cabeceras, filtros, interruptores, vacíos), `Icono` (SVG propios), `Avatar`, `FortalezaClave`, `DialogoEnlace`, `Paginacion`.
`services/`	`clienteApi` (fetch + renovación de sesión), `sesionApi`, `adminApi`, `consultasApi`, `reproductorApi`, `almacenSesion` (sessionStorage), `almacenReproductor` (localStorage y Cache Storage), `almacenTema`.
`hooks/`	`useDatos` / `useAccion` (carga y mutaciones), `useAuth`, `useAvisos` (toasts), `useTema`, `useSincronizacionReproductor`, `useFotoUsuario`, `useMedio`.
`context/`	`ProveedorAuth` (sesión) y `ProveedorAvisos`.
`types/api.ts`	Tipos de la API escritos a mano (espejo de los DTO).
`constants/`, `utils/`	Límites y roles; formato de fechas y tamaños, claves, descarga/CSV, estimación de reproducción.

8.2 Rutas

Ruta	Pantalla	Acceso
`/iniciar-sesion`	Inicio de sesión (indica si el servidor está disponible)	Público
`/registro`	Creación del primer administrador	Público (solo sin usuarios)
`/activar?token=…`	Activación de cuenta por invitación	Público
`/reproductor`	Reproductor web (`?token=` la primera vez)	Token del dispositivo
`/admin/supervision` (índice)	Supervisión	Autenticado
`/admin/sucursales`, `/pantallas`, `/multimedia`, `/listas`, `/historial`, `/cuenta`	Resto del panel	Autenticado
`/admin/usuarios`, `/admin/auditoria`	Usuarios y auditoría	Solo administrador

La interfaz oculta lo que el rol no puede hacer, pero **la autoridad es siempre la API**: los controles del cliente son de comodidad.

8.3 Patrones del cliente que hay que respetar

- **Sesión:** los tokens van en `sessionStorage` (clave `adplay.sesion`). Ante un 401, el cliente renueva *una sola vez a la vez* (se comparte la promesa) y reintenta; si falla, notifica expiración y vuelve al inicio de sesión.
- **Carga de datos:** `useDatos` para lecturas y `useAccion` para mutaciones, con mensajes de error en español tomados del `ProblemDetails`.
- **Tema:** clase `.claro` / `.oscuro` en `<html>`, aplicada antes de pintar (script en `index.html`) y guardada en `localStorage` (`adplay-tema`). Los colores son variables CSS; no use colores sueltos.
- **Lint estricto:** `eslint-plugin-react-hooks` prohíbe reasignar variables durante el render; calcule con `reduce` / `Map`.
- **URL de la API:** `VITE_API_BASE_URL` es obligatoria; sin ella la aplicación lanza un error al arrancar.
- **Sondeos:** Supervisión se refresca cada 30 s y la barra superior consulta el estado de red cada 30 s.

9. Reproductor y paquete local

9.1 Reproductor web (`/reproductor`)

1. Lee `?token=` una vez, lo guarda en `localStorage` y lo quita de la barra de direcciones.
2. Arranca sin conexión si puede: usa el último manifiesto guardado y los medios en **Cache Storage** (`adplay-medios`, claves por hash).
3. Cada *intervalo* (30 s, lo dicta el manifiesto) pide el manifiesto con `If-None-Match`; el servidor responde **304** si nada cambió (ETag = `"{version}-{habilitada}"`).
4. Si hay versión nueva, descarga **todos** los medios que falten, verifica el SHA-256 (si hay contexto seguro) y solo entonces cambia de versión; si algo falla mantiene la anterior.
5. Reproduce en bucle: imágenes por duración; videos hasta que terminan (mudos, `autoplay`). Un medio que falla se omite tras 1 s y se reporta.
6. Envía un **latido** por ciclo con versión cargada, último error y `{posicion, anuncioId, segundo}`.
7. Un 401 (token revocado) muestra «Pantalla no autorizada». Pantalla completa al primer clic.

El panel estima el avance entre latidos con `utils/reproduccion.ts` (el dato real solo llega cada ciclo).

9.2 Paquete ZIP para reproductor local

```
AdPlay_{Lista}_v{N}.zip
├── manifest.json     { formatoManifiesto:1, version, publicacionId, numeroVersion, nombreLista,
│                       generadoUtc, intervaloConsultaSegundos,
│                       elementos:[{ posicion, anuncioId, titulo, archivoMultimediaId, nombreArchivo,
│                                    ruta:"medios/001_ab12cd34ef56.jpg", tipoMultimedia, tipoMime,
│                                    duracionSegundos, tamanoBytes, hashSha256 }] }
├── version.txt       resumen legible
└── medios/           001_{hash12}.ext, 002_… (ordenados por posición; sin compresión)
```

> **Brecha principal frente al plan.** El formato está definido (`FormatoPaquete`) y el backend lo genera y verifica, y la guía prevé que una persona autorizada lo descargue y lo coloque en la carpeta del **sistema de reproducción local que la empresa ya utiliza**. Lo que **no está confirmado** es que ese sistema entienda el `manifest.json` (nombres, orden y duración); si no lo entiende, habrá que adaptar el formato de exportación a lo que ese sistema espera. Tampoco hay automatización de la copia (requeriría un agente autorizado por TI). Hoy el modo «Paquete local» de una pantalla marca la preferencia y permite descargar el ZIP. Resolver las preguntas de la «Fase 0» (sección 16.2) antes de invertir más aquí.

10. Configuración, ejecución y despliegue

10.1 Requisitos y primer arranque

.NET 10 SDK, Node.js y una instancia accesible de SQL Server. La API usa el puerto **5218** y el panel el **5173**.

```
# 1. Cadena de conexión (no está en el repositorio; se guarda con User Secrets)
dotnet user-secrets set "ConnectionStrings:AdPlay"
"Server=localhost;Database=EDEEsteAdPlay;Trusted_Connection=True;TrustServerCertificate=True" --project
EDEEste.AdPlay.API

# 2. API (en Development aplica las migraciones al iniciar)
dotnet run --project EDEEste.AdPlay.API --launch-profile http

# 3. Panel
cd EDEEste.AdPlay.Presentation
npm install
npm run dev          # http://localhost:5173  (VITE_API_BASE_URL viene en .env.development)
```

Primera cuenta: con la base vacía, `/iniciar-sesion` ofrece «Cree la primera cuenta» (`/registro`), que crea al primer administrador; después ese registro se cierra. Alternativa: definir `Seguridad:AdministradorInicial:Correo` y `:Clave` en User Secrets (en `Development` la inicialización ya está activa).

10.2 Configuración

Clave	Predeterminado	Efecto
`ConnectionStrings:AdPlay`	— (obligatoria)	Sin ella la API no arranca y explica cómo configurarla.
`BaseDatos:AplicarMigracionesAlIniciar`	false (true en Development)	Aplica migraciones pendientes al arrancar.
`Almacenamiento:RutaRaiz`	`App_Data/Almacenamiento`	Carpeta de medios, fotos y paquetes. Ruta relativa = desde la raíz del proyecto API.
`Multimedia:TamanoMaximoImagenMb / VideoMb / FotoPerfilMb`	20 / 500 / 2	Límites de carga.
`Reproductor:IntervaloConsultaSegundos`	30	Cada cuánto consulta la pantalla.
`Reproductor:IntervalosParaDesconexion`	3	Intervalos sin latido para marcar «desconectada» (ventana = 90 s).
`Cors:OrigenesPermitidos`	[ ] (Dev: localhost:5173)	Orígenes del panel. Vacío = el panel no podrá llamar a la API.
`Seguridad:DuracionAccesoMinutos / DuracionRenovacionHoras`	30 / 12	Vida de los tokens de sesión.
`Seguridad:IntentosInicioSesionPorMinuto`	10	Límite por IP.

Clave	Predeterminado	Efecto
`Seguridad:InicializarAlIniciar`	false (true en Development)	Crea roles y, si no hay usuarios, el administrador configurado.
`VITE_API_BASE_URL` (frontend)	— (obligatoria)	URL pública de la API; **se incorpora al JavaScript en la compilación**.

10.3 Despliegue

> **No hay despliegue definido.** El repositorio no incluye Dockerfile, pipeline de CI/CD, scripts de publicación ni documentación de infraestructura. Lo siguiente es lo que habrá que decidir y resolver.

- **Frontend:** `npm run build` genera archivos estáticos en `dist/` (servibles con IIS, Nginx o un contenedor). La URL de la API se fija al compilar: una compilación por ambiente.
- **API:** aplicación ASP.NET Core estándar. Detrás de un proxy inverso, configure `ForwardedHeaders` (hoy no está en el código) para que el límite de intentos vea la IP real; fuerce HTTPS y agregue HSTS.
- **Base de datos:** aplique las migraciones con un script idempotente como paso del despliegue; defina respaldos.
- **Almacenamiento:** un directorio o recurso compartido con permisos restringidos y **respaldo**; es donde viven los medios y paquetes (la base solo guarda sus metadatos).
- **Sesiones:** persista las llaves de Data Protection (hallazgo S-09); si no, cada reinicio cierra todas las sesiones y varias instancias no se entienden.
- **Zona horaria:** los horarios usan identificadores IANA (`America/Santo_Domingo`). En Linux/contenedores hace falta `tzdata` e ICU.
- **Primer despliegue:** el registro de la primera cuenta es público mientras no haya usuarios (hallazgo S-14): cree el administrador por configuración *antes* de exponer el servicio.

10.4 Problemas frecuentes

Síntoma	Causa probable
«No se pudo conectar con el servidor»	La API no está en ejecución, el puerto/URL no coincide con `VITE_API_BASE_URL`, o el origen del panel no está en `Cors:OrigenesPermitidos`.
La API no arranca: falta la cadena de conexión	No se configuró el User Secret; el mensaje indica el comando.
Error de SQL al arrancar tras bajar cambios	Hay migraciones nuevas y el ambiente no las aplica solo (fuera de Development): ejecute `dotnet ef database update`.
Todas las sesiones se cierran al reiniciar la API	Las llaves de Data Protection no están persistidas (esperado hoy).
«Demasiados intentos»	Límite de 10/min por IP, o cuenta bloqueada 10 min tras 5 fallos (un administrador puede restablecer la contraseña y levanta el bloqueo).
El video no se reproduce en el televisor	El códec del MP4 no lo soporta el navegador del equipo; se acepta cualquier MP4/WebM, no se transcodifica.

11. Pruebas y calidad

Área	Qué cubre	Casos
Domain	Asignaciones (destino único, vigencia, ambigüedad), horarios (franjas, zonas, solapamiento), jerarquía de grupos, resolución de prioridad, lista, estado de pantalla.	37 métodos
Application	Autenticación, usuarios (último administrador), invitaciones, foto de perfil, validador de archivos (extensión, MIME, firma, tamaño).	32 métodos
Infrastructure	Modelo de EF Core: longitudes, CHECK, FKs y enums sin base de datos real.	4 métodos
Total	xUnit, dobles manuales. Resultado actual: **98 pruebas, todas pasan.**	73 métodos (98 con casos de *Theory*)

```
dotnet test
npm run lint  --prefix EDEEste.AdPlay.Presentation
npm run build --prefix EDEEste.AdPlay.Presentation   # incluye tsc -b
```

Lo que no está cubierto

- **Servicios con repositorios:** publicaciones, reproductor, reportes, exportaciones, multimedia, asignaciones, sucursales, pantallas e historial no tienen pruebas de servicio.
- **API:** ninguna prueba de integración (`WebApplicationFactory`): políticas de autorización, serialización, ETag/304, rate limiting y manejo de errores solo se han probado a mano.
- **Base de datos real:** consultas EF, índices únicos filtrados, cascadas y migraciones no se ejercitan en pruebas.
- **Frontend:** ni pruebas unitarias ni de extremo a extremo; la calidad descansa en TypeScript estricto y lint.
- **Reproductor:** la sincronización (caché, hash, cambio de versión, sin conexión) no tiene pruebas automatizadas.

12. Revisión de seguridad: hallazgos

> **Alcance y método.** Revisión manual del código fuente (API, aplicación, infraestructura y panel), de la configuración y de las migraciones, más el análisis de dependencias del 14/10/2026 (`dotnet list package --vulnerable --include-transitive` y `npm audit`). **No** incluye pruebas de penetración dinámicas, revisión de infraestructura (servidor, red, TLS, respaldos) ni modelado formal de amenazas. Las severidades son una estimación para el contexto actual (sistema interno, pocos usuarios, sin despliegue productivo todavía).

ID	Severidad	Hallazgo	Remediación
S-01	ALTA	**Dependencia transitiva vulnerable.** `System.Security.Cryptography.Pkcs` 9.0.11 (llega por `Microsoft.EntityFrameworkCore.SqlServer → Microsoft.Data.SqlClient 6.1.6`) con avisos **GHSA-846f-f764-76mx (alta)** y GHSA-2fmv-rv2c-59x7 (moderada). Afecta API, Infrastructure y pruebas. El código no usa PKCS directamente, por lo que la explotabilidad probable es baja, pero la alerta es de alta severidad.	Actualizar `Microsoft.Data.SqlClient`/EF Core a una versión que arrastre la corrección, o agregar un `PackageReference` explícito a `System.Security.Cryptography.Pkcs` en la versión corregida que indican los avisos. Repetir `dotnet list package --vulnerable` y dejarlo como paso de CI.
S-02	MEDIA	**Sesión sin revocación ni tope absoluto.** Los tokens son sin estado: no existe «cerrar sesión» en el servidor; cambiar o restablecer la contraseña no invalida los tokens vigentes; el token de acceso de un usuario desactivado sigue sirviendo hasta 30 min; y cada renovación emite un nuevo token de renovación de 12 h, por lo que la sesión puede prolongarse indefinidamente (`Results.SignIn` en `AutenticacionEndpoints.cs`). Un token de renovación robado sirve 12 h y se puede encadenar.	Validar el *security stamp* del usuario en cada petición (o al renovar y reducir la vida del acceso); registrar el momento de inicio de sesión en el token y exigir un tope absoluto; agregar `POST /auth/cerrar-sesion` con revocación (lista de `jti` o tokens de renovación persistidos y rotativos).
S-03	MEDIA	**Sin cabeceras de seguridad HTTP.** No se configuran `Content-Security-Policy`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `frame-ancestors`/`X-Frame-Options` ni HSTS (`UseHsts` no se llama; solo `UseHttpsRedirection` fuera de Development). Los medios y fotos se sirven con el MIME guardado y sin *nosniff* (mitiga la lista blanca de formatos).	Middleware de cabeceras en la API y en el servidor que sirva `dist/`; CSP estricta (`default-src 'self'`, permitir solo la API y los `blob:` del reproductor); HSTS en producción.

ID	Severidad	Hallazgo	Remediación
S-04	MEDIA	**El token de la pantalla viaja en la URL y no caduca.** El enlace `/reproductor?token=…` queda en el historial del navegador, en el portapapeles y en cualquier registro intermedio; el cliente lo quita de la barra, pero la primera carga ya lo envió al servidor de archivos. Se guarda en `localStorage` sin vencimiento ni rotación automática.	Entregarlo en el fragmento (`#token=…` , que no se envía al servidor ni en `Referer`) o canjear un código corto de un solo uso por el token; definir caducidad/rotación; mostrar en el panel la última dirección IP/fecha de uso.
S-05	MEDIA	**Contraseñas pegadas en el campo de correo quedan en la auditoría.** Un inicio de sesión fallido guarda `Correo: {texto escrito}` (hasta 256 caracteres) en `RegistrosAuditoria.Detalle` (`ServicioAutenticacion.cs`); si la persona escribe su contraseña en ese campo, queda en claro y visible para los administradores.	Registrar el texto solo si es una dirección de correo válida; si no, guardar un marcador («entrada no válida») y la longitud.
S-06	MEDIA	**Límites de intentos insuficientes.** El limitador es solo por IP (detrás de un proxy todos comparten IP: `ForwardedHeaders` no está configurado en el código), no cubre al reproductor ni a las demás rutas, y el bloqueo de 5 intentos permite que un atacante deje fuera a una cuenta a propósito (bloqueo dirigido de un administrador).	Configurar `ForwardedHeaders` con proxies de confianza; limitador por IP *y* por cuenta con espera progresiva; límites razonables en el reproductor y en las cargas; alertar de bloqueos repetidos.
S-07	MEDIA	**Archivos subidos sin análisis de contenido.** Se valida extensión, MIME y firma inicial, pero no se inspecciona ni se re-codifica el contenido (MP4: solo la marca `ftyp`); no hay antivirus. La duración del video la declara el cliente. Los archivos se entregan a todos los televisores y navegadores.	Procesar en el servidor con `ffprobe` / `ffmpeg` (metadatos reales, transcodificar a un perfil conocido, generar miniatura) en un proceso aislado; considerar escaneo antivirus del directorio de carga.
S-08	MEDIA	**Exportación de paquetes sin tope ni retención.** Cada solicitud genera un ZIP nuevo, de forma **síncrona** dentro de la petición HTTP y usando el temporal del sistema; no se reutiliza uno existente ni se purgan antiguos. Un Operador (o un script con su sesión) puede agotar disco, tiempo de petición y memoria.	Reutilizar la exportación completada de la misma publicación; ejecutar en segundo plano con cola y límite de concurrencia; política de retención y purga; cuota por usuario.
S-09	MEDIA	**Llaves de Data Protection sin persistir.** Los tokens de sesión se protegen con ellas; en cada reinicio (o con más de una instancia) las sesiones dejan de ser válidas. También las usa la renovación.	`PersistKeysToFileSystem` / `DbContext` /almacén compartido, con protección de las llaves en reposo; fijar `SetApplicationName` .

ID	Severidad	Hallazgo	Remediación
S-10	MEDIA	**Ventana de toma de control en el primer despliegue.** `/registro` es público mientras la base no tenga usuarios; quien llegue primero a un servicio recién desplegado y accesible se convierte en administrador.	Crear al administrador por configuración antes de exponer el servicio, o exigir un código de configuración inicial (variable de entorno / consola) para el registro; documentarlo en el procedimiento de despliegue.
S-11	BAJA	**Exclusión mutua solo dentro de un proceso.** El registro inicial y la activación de invitaciones usan un `SemaphoreSlim` estático; con más de una instancia puede haber carrera. La unicidad por `UserName` en la base la mitiga (la segunda falla), pero no devuelve un error limpio.	Apoyarse en las restricciones únicas y manejar `DbUpdateException` como 409; o bloqueo distribuido si se escala.
S-12	BAJA	**El mensaje de bloqueo revela que la cuenta existe.** «Demasiados intentos fallidos…» solo aparece para cuentas reales; el resto devuelve el mensaje genérico.	Devolver siempre el mensaje genérico y comunicar el bloqueo por otro canal (correo).
S-13	BAJA	**Superficie del cliente.** El token de sesión está en `sessionStorage` (legible por XSS; hoy no hay vectores conocidos y React escapa por defecto, pero sin CSP no hay segunda barrera) y las fuentes se piden a Google Fonts (dependencia externa en la ruta crítica; en redes sin salida a Internet el diseño cae a tipografías de respaldo, y se revela la IP del equipo).	CSP estricta; autoalojar las fuentes (hay archivos libres: Charis SIL y Sansation); a mediano plazo evaluar cookies `HttpOnly` + `SameSite` con un BFF.
S-14	BAJA	**Configuración permisiva por defecto.** `AllowedHosts: "*"`; `Cors:OrigenesPermitidos` vacío en `appsettings.json` (falla cerrado, pero silenciosamente).	Fijar `AllowedHosts` por ambiente; validar al arrancar que CORS esté configurado y avisar en el registro.

Resultado del análisis de dependencias

Ecosistema	Resultado
NuGet (`dotnet list package --vulnerable --include-transitive`)	**1 paquete afectado** (S-01). Domain y Application: sin vulnerabilidades.
npm (`npm audit`, producción y desarrollo)	0 vulnerabilidades.

> **Lo que se revisó y está bien:** toda ruta nueva queda protegida por defecto (`FallbackPolicy`); las operaciones sensibles exigen `Administracion`; no hay SQL construido a mano (todo EF/LINQ); los tokens de dispositivo e invitación se guardan hasheados; no hay endpoints que acepten rutas de archivo del cliente; el reproductor solo accede a lo suyo; los errores no filtran trazas; no hay secretos en el repositorio ni en el historial reciente (el `.http` trae marcadores y los `.env` no llevan secretos).

13. Deuda técnica y prácticas a mejorar

Cosas que funcionan hoy pero que están de modo básico, o que no son buenas prácticas y conviene corregir antes de crecer.

ID	Tema	Detalle y propuesta
D-01	**Páginas del panel demasiado grandes**	`PantallasPage` (916 líneas), `MultimediaPage` (812), `UsuariosPage` (641), `CuentaPage` (572), `SeccionAsignaciones` (503). Dividir en componentes y hooks por responsabilidad (formulario, tabla, diálogos).
D-02	**Estilos en línea y duplicación**	Abundan `style={{…}}` mezclados con CSS por página. Constantes repetidas (p. ej. descripciones de roles en `UsuariosPage` y `CuentaPage`). Llevar a clases/tokens y a `constants/`.
D-03	**Tipos del cliente escritos a mano**	`types/api.ts` duplica los DTO de C#: riesgo de divergencia silenciosa. Publicar OpenAPI (hoy solo en Development) y generar el cliente tipado.
D-04	**Sin pruebas de integración ni del frontend**	Ver sección 11. Es la deuda de mayor impacto sobre la velocidad de cambio.
D-05	**Sin CI/CD, contenedores ni *health checks***	Falta pipeline (compilar, probar, lint, auditoría de dependencias), `Dockerfile`, `/health` y `/ready`.
D-06	**Observabilidad mínima**	Solo `ILogger` por consola. Falta registro estructurado (Serilog/OpenTelemetry), correlación por `traceId`, métricas (pantallas en línea, errores de reproducción, latencia) y alertas.
D-07	**Listados sin paginación**	Solo auditoría e historial paginan. Multimedia, anuncios, pantallas, listas, usuarios y el reporte de supervisión devuelven todo; el reporte recalcula todo en cada llamada y el panel lo pide cada 30 s por usuario (más la barra superior). Paginar/filtrar en servidor y cachear unos segundos.
D-08	**Carga repetida en el reproductor**	`ObtenerMedioAsync` resuelve asignaciones y publicación completa en *cada* descarga de medio. Con muchas pantallas y muchos medios multiplica consultas. Cachear por (token, versión) unos segundos.
D-09	**Sin control de concurrencia**	Dos personas editando la misma lista: gana el último. Dos publicaciones simultáneas pueden chocar con el índice único (`NumeroVersion`) y devolver 500 en lugar de 409. Agregar `rowversion` y manejar `DbUpdateConcurrencyException` / `DbUpdateException`.
D-10	**Exportación en la petición HTTP**	Ver S-08.
D-11	**Caché del reproductor sin depuración**	El Cache Storage `adplay-medios` acumula todos los medios que alguna vez descargó el televisor; nunca se limpia. Eliminar entradas que ya no estén en el manifiesto vigente.
D-12	**Metadatos de medios incompletos**	Ancho y alto no se calculan; la duración de video es manual; no hay miniatura de video (el panel usa degradados). Resolver junto con S-07.
D-13	**Versionado de la API**	Sin `/api/v1`. El formato del manifiesto sí lleva versión (`formatoManifiesto` = 1), lo cual es valioso para reproductores desplegados: manténgalo.

ID	Tema	Detalle y propuesta
D-14	**Accesibilidad no auditada**	Hay etiquetas `aria`, foco en diálogos y contraste en dos temas, pero no se ha validado con lector de pantalla ni con navegación solo teclado de punta a punta.
D-15	**Textos incrustados**	Todo el español está en el código (sin i18n). Aceptable hoy; relevante si se prevé otro idioma.
D-16	**Sin borrado funcional**	No se pueden eliminar sucursales, grupos, pantallas, listas ni anuncios (solo desactivar/archivar), ni medios. Es deliberado por la trazabilidad, pero hay que definir una política de retención/limpieza (datos de prueba, medios sin uso, paquetes antiguos).
D-17	**Retirar es irreversible**	Una versión retirada no se puede volver a publicar; hay que crear una nueva. Evaluar si es lo deseado.
D-18	**Falta guía de operación**	No hay runbook: respaldos, restauración, rotación de tokens de pantallas, alta/baja de sucursales, qué hacer cuando una pantalla no conecta.

14. Lista de pendientes

Consolida lo que declara el README, los vacíos funcionales y los hallazgos de las secciones 12 y 13.

14.1 Funcionales (negocio)

#	Pendiente	Origen	Tamaño
F1	**Integración con el sistema de reproducción local existente:** confirmar o adaptar el formato del paquete a lo que ese sistema espera y, si TI lo autoriza, un agente de sincronización que copie el paquete a su carpeta (y, si se aprueba, SignalR). Depende de las respuestas de la Fase 0.	README, guía	M–L
F2	**Correo saliente:** envío automático de invitaciones, recuperación de contraseña y alertas de pantalla desconectada.	README	M
F3	**Autenticación en dos pasos** o integración con el directorio corporativo (SSO).	README	L
F4	**Miniaturas de video y metadatos** (duración, ancho, alto) calculados en el servidor.	README	M
F5	**Historial técnico de estados de pantalla** (`HistorialEstadoPantalla`): línea de tiempo de conexiones y errores.	README (opcional)	M
F6	**Exportación de paquetes en segundo plano** con estado y reutilización.	README	M
F7	**Control de concurrencia** en la edición de listas y asignaciones.	README	S

14.2 Técnicos y de calidad

#	Pendiente	Origen	Tamaño
T1	Pruebas de servicio con repositorios simulados y pruebas de integración de la API contra SQL Server.	README, D-04	M
T2	Pruebas del frontend (Vitest + Testing Library) y de extremo a extremo (Playwright) de los flujos críticos.	D-04	M
T3	CI/CD, Dockerfile, *health checks* y script idempotente de migraciones.	D-05	M
T4	Registro estructurado, métricas y alertas.	D-06	M
T5	Paginación y caché de listados y del reporte de supervisión; caché de resolución en el reproductor.	D-07, D-08	M
T6	Generar el cliente TypeScript desde OpenAPI; partir las páginas grandes.	D-01, D-03	M
T7	Depurar la caché del reproductor.	D-11	S
T8	Todos los hallazgos S-01 a S-14 (sección 12).	Sección 12	S–M

Tamaño orientativo para una persona: S = hasta 2 días · M = hasta 2 semanas · L = más de 2 semanas (requiere decisiones previas).

15. Siguientes pasos recomendados

Orden sugerido. Cada fase tiene una meta verificable.

Fase A — Estabilizar y proteger (1 a 2 semanas)

Meta: un build que se puede confiar y un ambiente reproducible.

1. **S-01:** resolver la dependencia vulnerable y confirmar con `dotnet list package --vulnerable`.
2. **Probar el flujo de invitaciones y la migración** contra SQL Server real (crear invitación, abrir enlace, activar, anular, vencer) y corregir lo que aparezca.
3. **Quick wins de seguridad:** S-05 (no registrar texto que no sea correo), S-04 (token en fragmento `#`), S-03 (cabeceras y HSTS), S-14 (CORS/hosts), S-12 (mensaje genérico).
4. **CI mínimo (T3):** compilar, `dotnet test`, `npm run lint`, `npm run build`, `npm audit` y `dotnet list package --vulnerable` en cada cambio.
5. **Pruebas de integración (T1):** `WebApplicationFactory` sobre un SQL Server efímero para: políticas de autorización por endpoint, reproductor (manifiesto/304/medios/latido) y publicar/retirar.
6. **Recuperar la guía original** (`EDEEste.AdPlay_Guia_Arquitectura.pdf`) y archivarla en el repositorio o en el espacio del equipo.

Fase B — Endurecer y hacer operable (1 a 2 meses)

Meta: listo para un piloto en una sucursal.

1. **Sesión (S-02, S-09, S-06):** security stamp, cierre de sesión en servidor, tope absoluto, llaves persistidas, `ForwardedHeaders` y límites por cuenta.
2. **Registro inicial seguro (S-10)** y runbook de primer despliegue.
3. **Procesamiento de medios (S-07, F4):** `ffprobe` / `ffmpeg` para metadatos, transcodificación a un perfil compatible con los televisores y miniaturas.
4. **Exportaciones en segundo plano (S-08, F6)** con retención.
5. **Correo saliente (F2):** definir una interfaz `INotificador` en Application e implementarla con el SMTP/Graph corporativo; usarla en invitaciones y alertas.
6. **Observabilidad (T4):** registro estructurado, *health checks* y un tablero con pantallas en línea/desconectadas y errores.
7. **Despliegue (T3):** contenedores o IIS, secretos, respaldos de base y almacenamiento, procedimiento de migraciones, procedimiento de rotación de tokens de pantalla.
8. **Concurrencia (F7, D-09)** y paginación/caché (T5).

Fase C — Crecer (a partir del piloto)

Meta: cubrir lo que el plan condicionaba a la Fase 0.

1. **Cerrar el modo local (F1):** con las respuestas de la Fase 0 (sección 16.2), probar el paquete exportado contra el sistema local existente y ajustar el formato; después decidir entre dejar la entrega manual, un agente de sincronización autorizado por TI o usar solo el modo web con el navegador en modo kiosco y arranque automático.
2. **Tiempo real:** si se justifica, SignalR para empujar cambios y reducir la latencia de la consulta de 30 s.
3. **Identidad corporativa (F3):** SSO con el directorio y/o doble factor.
4. **Calidad del panel (T2, T6, D-14):** pruebas E2E, cliente generado, partir páginas grandes y auditoría de accesibilidad.
5. **Historial técnico y alertas (F5)** y reportes de cumplimiento (qué se emitió y cuándo), si negocio lo pide.

16. Anexos

16.1 Decisiones de diseño tomadas (no reabrir sin motivo)

- **Publicaciones y medios inmutables.** Es lo que permite saber exactamente qué vio cada pantalla y revertir sin pérdida.
- **Tokens y claves se guardan hasheados** (dispositivo e invitación); el valor en claro se muestra una sola vez.
- **No hay registro abierto.** Solo la primera cuenta; las demás las crea o invita un administrador.
- **Dos roles** (Administrador, Operador) y tres políticas; la matriz está en la sección 2.4.
- **Usuarios no se eliminan**; se desactivan. La auditoría los referencia.
- **El Dominio no conoce Identity**: usa ids enteros y `IGestorUsuarios`.
- **Prioridad de asignaciones:** pantalla > grupo (más cercano) > sucursal; sin dos asignaciones ambiguas en el mismo destino.
- **Un solo proyecto de frontend** para panel y reproductor, sin librería de UI ni de estado.

16.2 Preguntas abiertas (Fase 0)

La guía original (sección 13) lista seis decisiones por confirmar con la empresa; el README indica que las tres primeras siguen sin respuesta y condicionan el modo local y la sincronización en tiempo real:

Pregunta de la guía	Por qué importa
¿Dónde está exactamente la carpeta que recibe el contenido actual (equipo local, televisor, PC conectado, recurso de red)?	Decide si el paquete solo se descarga, se copia a un recurso compartido o necesita un agente.
¿El orden y la duración vienen de nombres de archivo, de un archivo de configuración o de la app existente?	Define el formato real del manifiesto y la compatibilidad de la exportación.
¿La empresa permite un agente en un PC/servidor para sincronizar carpetas?	Necesario solo para automatizar la copia fuera del navegador.
¿Qué nivel de actualización se requiere?	Confirma si la consulta cada 30 s es suficiente o hace falta SignalR.
¿Qué formatos de imagen/video y tamaños son aceptados?	Afecta validación, reproducción en el navegador y empaquetado.
¿Cuántas pantallas y qué modelos/navegadores hay?	Dimensiona la consulta periódica y valida compatibilidad (pantalla completa, suspensión, códecs).

Preguntas adicionales que surgen de este análisis: ¿dónde se alojará (servidor propio, nube)? ¿Cuántas pantallas y sucursales en 12 meses? ¿Hay SMTP o Microsoft 365 para correo saliente? ¿Directorio corporativo para SSO? ¿Política de retención de medios y auditoría? ¿Qué navegador y modelo de televisores se usan (códecs de video soportados)?

16.3 Glosario

Término	Significado
Anuncio	Medio (imagen o video) con título y duración predeterminada.
Lista (de reproducción)	Secuencia editable de anuncios.
Publicación / versión	Instantánea inmutable de una lista (v1, v2…); es lo que reproducen las pantallas.
Asignación	Vínculo entre una lista y un destino (sucursal, grupo o pantalla), con vigencia y horarios.
Manifiesto	Documento JSON con la versión y los medios que una pantalla debe reproducir.
Latido	Señal periódica de la pantalla con su versión, error y anuncio en curso.
Token de dispositivo / enlace	Credencial de una pantalla; se entrega dentro de una URL del reproductor.
Paquete local	ZIP exportado de una versión para reproducción sin conexión.
Invitación	Enlace de un solo uso para que una persona active su cuenta.

16.4 Mapa rápido de archivos clave

Archivo	Por qué importa
`API/Program.cs`, `API/Configuracion/*`	Arranque, CORS, seguridad, errores, límites de carga.
`API/Seguridad/PoliticasAcceso.cs`	Políticas y roles.
`API/Endpoints/*.cs`	Superficie HTTP.
`Domain/Services/ResolutorAsignaciones.cs`, `Domain/Entities/ReglaHorario.cs`	Qué reproduce cada pantalla y cuándo.
`Application/CasosDeUso/Reproductor/ServicioReproductor.cs`	Contrato con las pantallas.
`Application/CasosDeUso/Publicaciones/ConstructorManifiesto.cs`	Formato del manifiesto y del paquete.
`Infrastructure/InyeccionDependencias.cs`	Política de contraseñas y bloqueo.
`Infrastructure/Persistencia/ContextoPublicidad.cs`	Modelo, convenciones, FKs a usuarios.
`Presentation/src/services/clienteApi.ts`	Cliente HTTP, renovación de sesión, cargas con progreso.
`Presentation/src/hooks/useSincronizacionReproductor.ts`	Motor del reproductor.
`Presentation/src/index.css`	Tokens de diseño y temas.
`README.md`	Resumen técnico vivo; manténgalo al día.